Dokumentation Techradar

Inhaltsverzeichnis

1. Einführung und Ziele . . . 2
   1.1. Aufgabenstellung . . . 2
   1.2. Qualitätsziele . . . 4
   1.3. Stakeholder . . . 4
2. Randbedingungen . . . 6
3. Kontextabgrenzung . . . 7
   3.1. Fachlicher Kontext . . . 7
   3.2. Technischer Kontext . . . 7
4. Lösungsstrategie . . . 9
5. Bausteinsicht . . . 10
   5.1. Level 2: Container-Diagramm . . . 10
   5.2. Level 3: Component-Diagramme . . . 10
6. Laufzeitsicht . . . 13
7. Verteilungssicht . . . 15
8. Querschnittliche Konzepte . . . 17
   8.1. API-Design . . . 17
   8.2. Testkonzept . . . 17
9. Architekturentscheidungen . . . 18
10. Risiken und technische Schulden . . . 19
   10.1. Risiken . . . 19
   10.2. Technische Schulden . . . 19
11. Fazit . . . 20
12. Arbeitsjournal . . . 22

1. Einführung und Ziele

1.1. Aufgabenstellung

1.1.1. User Story 1: Anmelden Technologie-Radar-Administration (Prio 'Could')

Als CTO oder Tech-Lead kann ich mich an der Technologie-Radar-Administration anmelden, damit ich Technologien im Technologie-Radar verwalten kann.

Akzeptanzkriterien

- Mittels korrektem Benutzername (E-Mail Adresse) und korrektem Passwort gelange ich zur Technologie-Radar-Administration.
- Mit nicht korrekten Benutzerdaten (Benutzername / Passwort) kann ich mich nicht anmelden.
- Nur Personen mit der Rolle CTO oder Tech-Lead dürfen sich in der Administration anmelden.

1.1.2. User Story 2: Technologie erfassen (Prio 'Must')

Als CTO kann ich eine neue Technologie erfassen, damit ich diese anschliessend publizieren kann.

Akzeptanzkriterien

- Für die Erfassung einer neuen Technologie muss ich folgende Felder angeben können:
  - Name (Muss-Feld) → z.B. **ArgoCD**
  - Kategorie (Muss-Feld) → z.B. **Tools**
  - Ring (Muss-Feld) → z.B. **Trial**
  - Beschreibung Technologie (Muss-Feld) → z.B. **Argo CD is a declarative, GitOps continuous delivery tool for Kubernetes.**
  - Beschreibung der Einordnung (Muss-Feld) → z.B. **Without making a judgment of the GitOps technique, we'd like to talk about Argo CD within the scope of deploying and monitoring applications in Kubernetes environments...**
- Sofern ein Muss-Feld nicht abgefüllt ist, darf die Technologie nicht abgespeichert werden.
- Neben den Eingaben müssen folgende Informationen bei der Erfassung einer neuen Technologie hinterlegt werden
  - Erfassungsdatum

1.1.3. User Story 3: Technologie entwerfen und später publizieren (Prio 'Should')

Als CTO kann ich erfasste, aber noch nicht publizierte, Technologien publizieren, damit diese auf dem Technologie-Radar erscheinen und entsprechned für Mitarbeiter einsehbar werden.

Akzeptanzkriterien

- Erfasste, aber noch nicht publizierte Technologien werden speziell gekennzeichnet.
- Für Entwürfe dürfen folgende Pflichtfelder leer sein:
  - Einordnung / Ring
  - Beschreibung der Einordnung
- Mit Klick auf **Publizieren** werden neue Technologien publiziert. Bei der Publizierung müssen folgende Eingaben gemacht werden:
  - Einordnung / Ring (Muss-Feld) → z.B. **Trial**
  - Beschreibung der Einordnung (Muss-Feld) → z.B. **Without making a judgment of the GitOps technique, we'd like to talk about Argo CD within the scope of deploying and monitoring applications in Kubernetes environments...**
- Neben den Eingaben müssen folgende Informationen bei der Publikation einer Technologie hinterlegt werden
  - Publikationsdatum.

1.1.4. User Story 4: Technologie ändern (Prio 'Should')

Als CTO kann ich erfasste Technologien ändern, um z.B. fehlerhafte Daten oder nicht mehr passende Informationen zu ändern.

Akzeptanzkriterien

- Folgende Felder können geändert werden:
  - Name (Muss-Feld)
  - Kategorie (Muss-Feld)
  - Beschreibung Technologie (Muss-Feld)
- Mit dem Speichern wird das Änderungsdatum der Technologie hinterlegt.

1.1.5. User Story 5: Technologie-Einordnung ändern (Prio 'Should')

Als CTO möchte ich die Einordnung von einer Technologie ändern, um die Relevanz der Technologie für das Unternehmen anders einstufen zu können.

Akzeptanzkriterien

- Mit Klick auf "Einordnung ändern", können folgende Felder einer Technologie geändert werden:
  - Einordnung / Ring (Muss-Feld) → z.B. **Adopt**
  - Beschreibung der neuen Einordnung (Muss-Feld)
- Mit dem Speichern wird das Änderungsdatum der Technologie hinterlegt.

1.1.6. User Story 6: Technologien anzeigen (Prio 'Must')

Als Mitarbeiter kann ich im Technologie-Radar-Viewer die Technologien resp. der Technologie-

Radar einsehen.

Akzeptanzkriterien

- Es werden nur publizierte Technologien angezeigt.
- Technologien werden nach ihren Kategorien (**Techniques**, **Platforms**, **Tools**, **Languages & Frameworks**) strukturiert.
- Technologien werden nach ihrer Maturität (**Assess**, **Trial**, **Adopt**, **Hold**) strukturiert.

Bemerkung: Die Technologien müssen nicht als **Radar** abgebildet werden, sondern können auch tabellarisch dargestellt werden.

1.1.7. User Story 7: Anmelden am Technologie-Radar-Viewer (Prio 'Could')

Als Mitarbeiter kann ich mich am Technologie-Radar-Viewer anmelden, damit ich die Technologien resp. der Technologie-Radar des Unternehmens einsehen kann.

Akzeptanzkriterien

- Mittels korrektem Benutzername (E-Mail Adresse) und korrektem Passwort gelange ich zum Technologie-Radar-Viewer.
- Mit nicht korrekten Benutzerdaten (Benutzername / Passwort) kann ich mich nicht anmelden.

1.2. Qualitätsziele

- Der **Technologie-Radar-Viewer** soll neben der Desktop-Ansicht, auch für die Mobile/Tablet-Ansicht optimiert sein.
- Der **Technologie-Radar-Viewer** soll bei einer 4G-Verbindung innerhalb 1s geladen werden.
  - Chrome Network-Throttling: *Fast 4G* Details
  - Others: 9 Mbit/s down, 1.5 Mbit/s up, 60ms Latency, 0% Package-Loss
- Sämtliche Anmeldungen an die **Technologie-Radar-Administration** werden aufgezeichnet.
- Die Funktionalitäten sollen mittels sinnvollen automatisierten **Unit/Integration-Tests** überprüft werden.

1.3. Stakeholder

Rolle	Kontakt	Erwartungshaltung
Auftraggeber	Dominik Witschard	Erwartet eine funktionsfähige Software, die die Anforderungen erfüllt und die Ziele erreicht.

Rolle	Kontakt	Erwartungshaltung
Entwickler	Mathias Vogel	Legt Wert darauf, dass das Projekt gut strukturiert ist, damit man sich im Code schnell zurechtfindet. Er erwartet, dass wichtige Abläufe durch Tests abgesichert sind, um bei Updates keine bestehenden Funktionen zu zerschiessen.

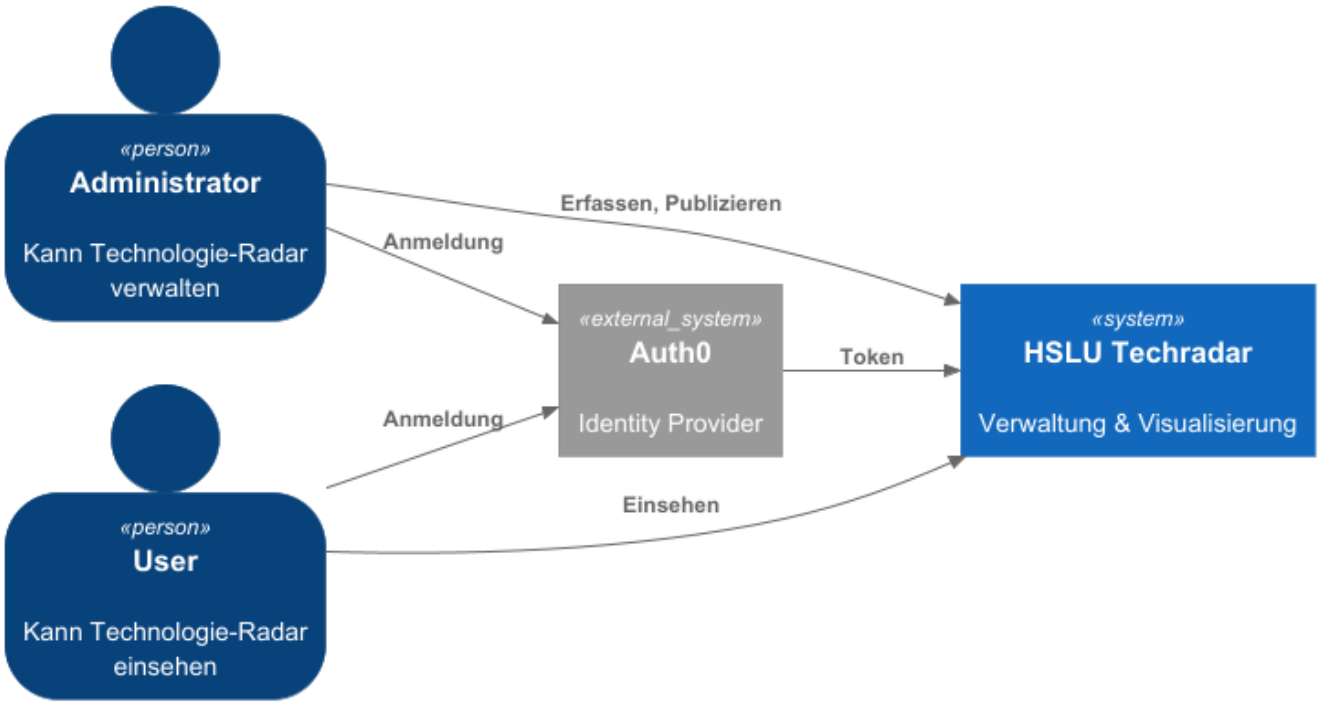
2. Randbedingungen

Die folgenden technischen und organisatorischen Vorgaben bilden den Rahmen für die Entwicklung des Technologie-Radars:

Typ	Bedingung	Erläuterung
Organisatorisch	**Einzelarbeit**	Das Projekt muss in Einzelarbeit ungesetzt werden.
Organisatorisch	**Aufwand (ca. 60h)**	Der gesamte Umfang für Implementierung und Dokumentation sollte ca. 60 Stunden betragen.
Organisatorisch	**Zeitplan**	Der Abgabetermin für das Projekt ist der 6. März.
Technisch	**Web-Stack**	Es soll eine Web-Applikation entwickelt werden, die Angular im Frontend und Node.js im Backend verwendet.
Technisch	**Datenpersistenz**	Die Daten müssen in irgendeiner Form persistiert werden.
Konvention	**Testabdeckung**	Sicherstellung der Qualität durch eine angemessene Abdeckung mit Unit- und E2E-Tests (Cypress).
Konvention	**Dokumentation**	Die Architektur muss dokumentiert werden.

3. Kontextabgrenzung

3.1. Fachlicher Kontext



Element	Beschreibung
Administrator	Ein CTO oder Tech-Lead, der die volle Berechtigung hat, neue Technologien im Radar zu erfassen und zu publizieren. Er meldet sich über Auth0 am System an.
User	Ein Mitarbeiter, der den Technologie-Radar einsehen kann, um sich über den aktuellen Stand der Technik im Unternehmen zu informieren. Auch er nutzt Auth0 für die Anmeldung.
Auth0	Ein externer Identity Provider, der die Authentifizierung (Login) übernimmt und dem Techradar-System nach erfolgreicher Anmeldung einen Token zur Rollenprüfung bereitstellt.
HSLU Techradar	Das Kernsystem, welches die Verwaltung der Technologiedaten durch Admins ermöglicht und die visualisierte Radar-Übersicht für alle Nutzer bereitstellt.

3.2. Technischer Kontext

Schnittstelle	Übertragungsweg	Protokoll / Format
Client (Browser) → Server	Internet / Intranet	HTTPS / REST (JSON)
Client (Browser) → Auth0	Externe API-Anbindung	HTTPS / OAuth 2.0 (OIDC)

Schnittstelle	Übertragungsweg	Protokoll / Format
Server → PostgreSQL-Datenbank	Internes Docker-Netzwerk	Prisma Client / TCP

4. Lösungsstrategie

Ziel / Anforderung	Lösungsansatz	Umsetzung
Client-Server-Architektur	Trennung von Frontend und Backend als eigenständige Applikationen	Angular (Client) kommuniziert via REST-API mit Express/Node.js. Typsicherheit durch OpenAPI-Generierung.
Datenpersistenz	Relationale Datenbank mit typsicherem ORM	PostgreSQL als Datenbank, Prisma ORM für Schema-Management, Migrationen und typsicheren Datenbankzugriff.
Authentifizierung & Autorisierung	Delegierte Authentifizierung über einen externen Identity Provider	Auth0 (OAuth 2.0 / OIDC) übernimmt Login und Rollenverwaltung (CTO/Mitarbeiter).
Responsive Darstellung	Mobile-/Tablet-optimierter Viewer	Tailwind CSS mit responsiven Breakpoints für Desktop und Mobile.
Testabdeckung	Qualitätssicherung durch automatisierte Tests auf mehreren Ebenen	E2E-Tests mit Cypress; Unit- und Integrationstests mit Vitest und Supertest.
Containerisierung	Reproduzierbare Entwicklungs- und Deployment-Umgebung	Docker Compose orchestriert Client, Server und Datenbank.
API-Dokumentation	Selbstdokumentierende API	Swagger UI generiert interaktive Dokumentation direkt aus dem Backend-Code.

5. Bausteinsicht

5.1. Level 2: Container-Diagramm

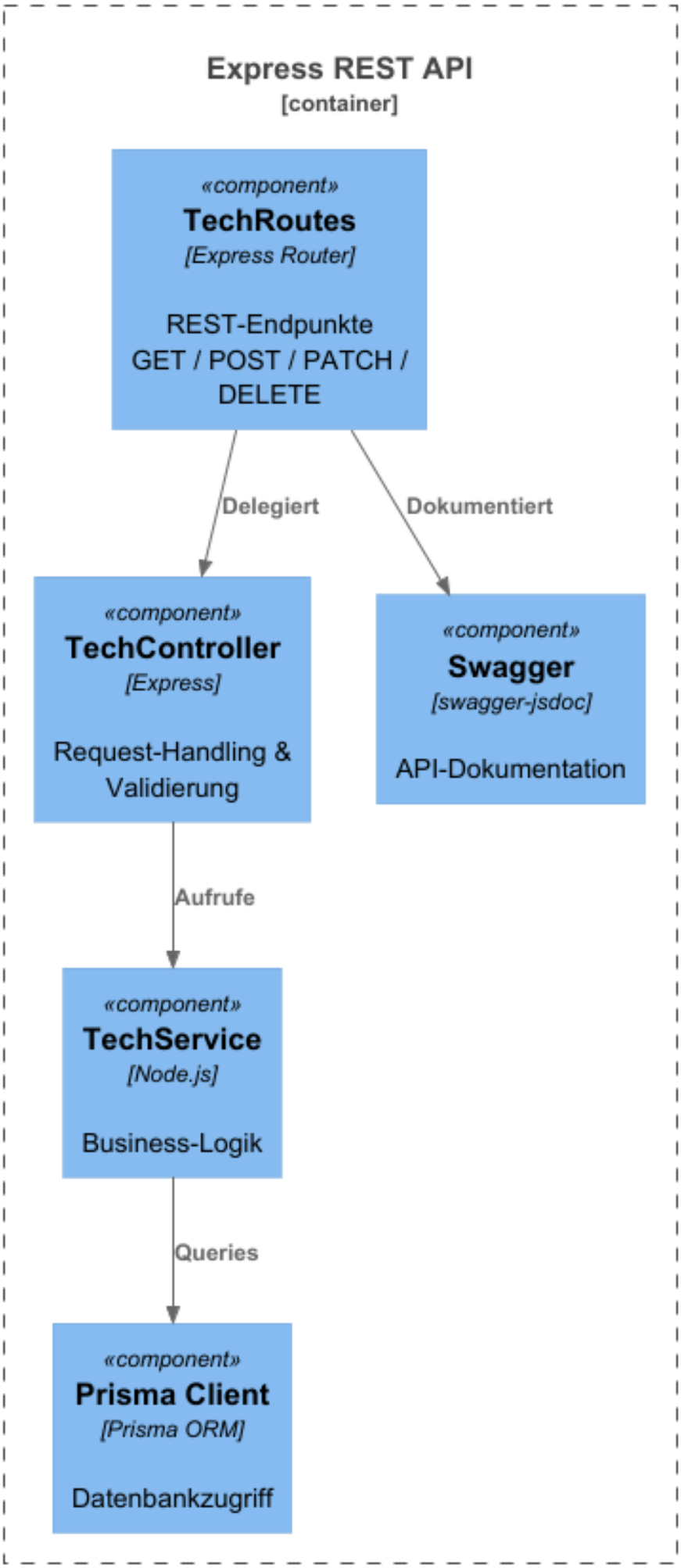
Element	Beschreibung
Angular SPA	Single-Page-Application für Radar-Visualisierung und Administration (Angular, TypeScript, Tailwind, DaisyUI).
REST API	Backend-Server mit REST-Endpunkten für die Verwaltung der Technologien (Express.js, Node.js).
Datenbank	PostgreSQL-Datenbank zur Persistierung der Technologiedaten.
Auth0	Externer Identity Provider für Authentifizierung via OAuth 2.0 / OIDC.

5.2. Level 3: Component-Diagramme

5.2.1. Client

Element	Beschreibung
NavbarComponent	Navigation zwischen den Seiten und Logout-Funktion.
HomeComponent	Startseite der Applikation.
RadarComponent	Visualisierung des Technologie-Radars mit publizierten Technologien.
RadarAdministrationComponent	Verwaltungsoberfläche für Technologien (Erstellen, Bearbeiten, Publizieren, Löschen).
TechFormComponent	Formular zum Erfassen, Bearbeiten und Publizieren von Technologien.
TechDetailComponent	Detailansicht einer einzelnen Technologie.
TechService	Angular Service für State-Management und API-Kommunikation.
OpenAPI Client	Typisierter HTTP-Client (`openapi-fetch`) für die Kommunikation mit der REST API.

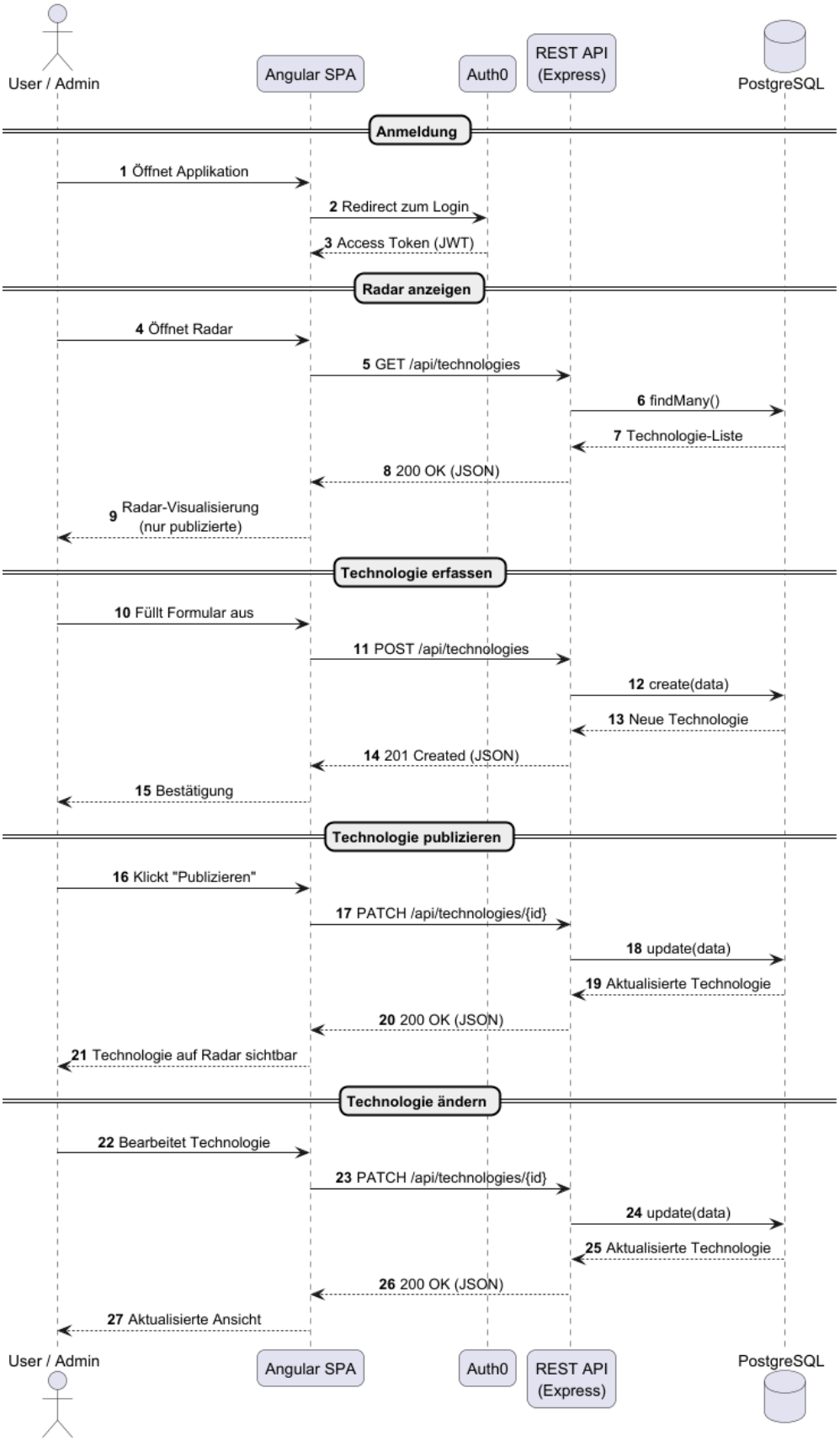
5.2.2. Server


Express REST API
[container]
«component»
TechRoutes
[Express Router]
REST-Endpunkte
GET / POST / PATCH /
DELETE
Delegiert
Dokumentiert
«component»
TechController
[Express]
Request-Handling &
Validierung
«component»
Swagger
[swagger-jsdoc]
API-Dokumentation
Aufrufe
«component»
TechService
[Node.js]
Business-Logik
Queries
«component»
Prisma Client
[Prisma ORM]
Datenbankzugriff

Element	Beschreibung
TechRoutes	Express Router mit REST-Endpunkten (GET, POST, PATCH, DELETE).
TechController	Verarbeitet eingehende Requests und validiert die Eingaben.
TechService	Enthält die Business-Logik für Technologie-Operationen.
Prisma Client	ORM-Schicht für den typsicheren Datenbankzugriff auf PostgreSQL.
Swagger	Generiert die interaktive API-Dokumentation aus den Route-Definitionen.

6. Laufzeitsicht

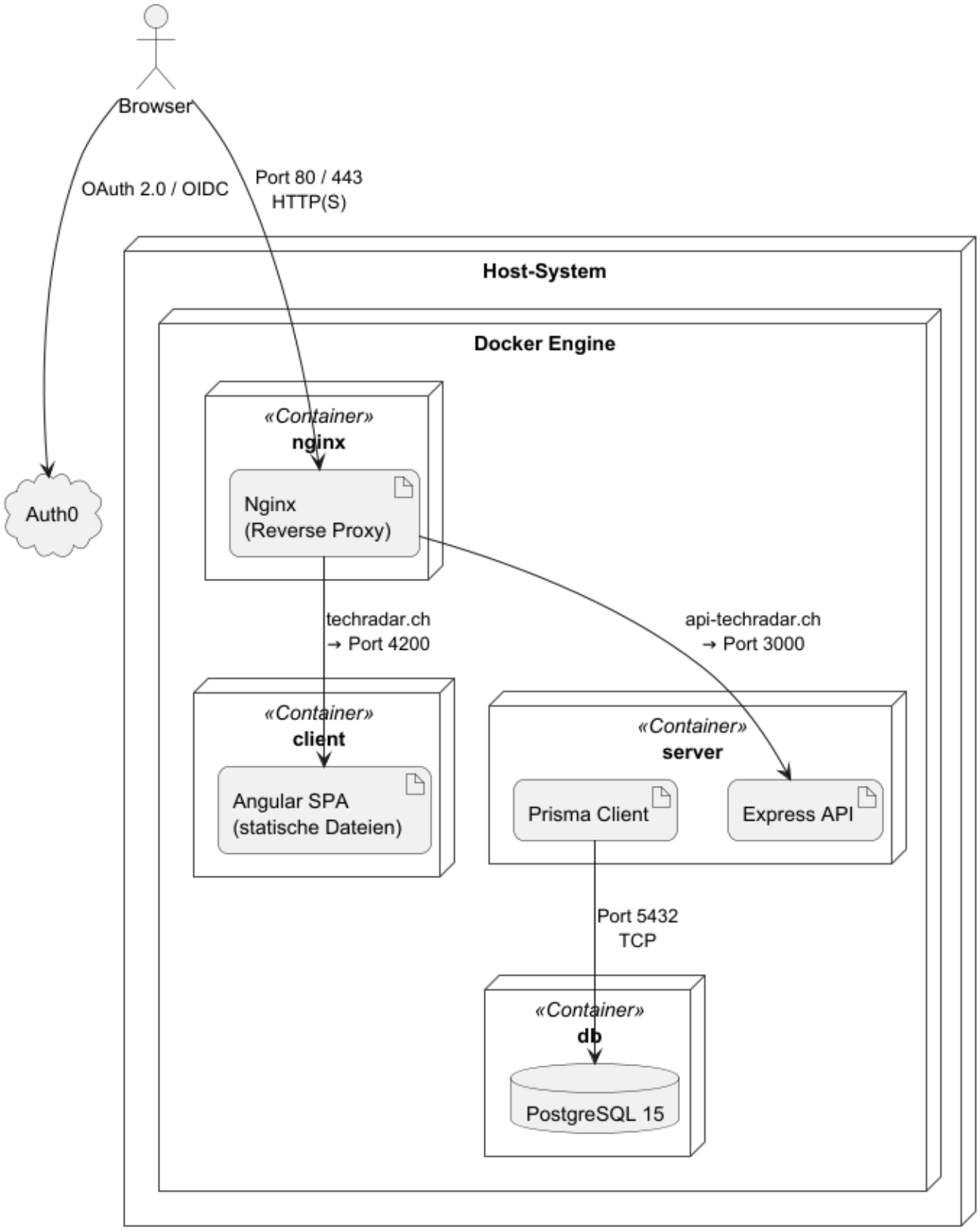
Das folgende Sequenzdiagramm zeigt die wichtigsten Laufzeitszenarien der Applikation:



Szenario	Beschreibung
Anmeldung	Der User öffnet die App, wird zu Auth0 weitergeleitet und erhält nach erfolgreichem Login ein JWT-Token.
Radar anzeigen	GET-Request an die API lädt alle Technologien. Der Client filtert und zeigt nur publizierte Einträge als Radar an.
Technologie erfassen	POST-Request sendet Formulardaten an die API, die via TechService einen neuen Eintrag in der Datenbank anlegt.
Technologie publizieren	PATCH-Request setzt `published: true` inkl. Ring und Begründung. Der TechService ergänzt automatisch das `publishedAt`-Datum.
Technologie ändern	PATCH-Request aktualisiert die gewünschten Felder einer bestehenden Technologie.

7. Verteilungssicht

Die Applikation wird mittels Docker Compose als vier Container betrieben. Ein separater Nginx Reverse Proxy ist der einzige Einstiegspunkt für den Browser und routet die Anfragen anhand der Domäne an die richtigen internen Container weiter.



Container	Image	Beschreibung
nginx	Nginx (Reverse Proxy)	Einziger Einstiegspunkt (Port 80/443). Routet anhand der Domäne an Client oder Server.
client	client	Liefert die gebaute Angular-Applikation als statische Dateien aus (Port 4200).
server	server	REST API mit Prisma ORM für die Technologie-Verwaltung (Port 3000).
db	PostgreSQL 15	Relationale Datenbank zur Persistierung der Technologiedaten (Port 5432).

8. Querschnittliche Konzepte

8.1. API-Design

Die API folgt REST-Prinzipien mit klaren Ressourcen (z.B. `/technologies`) und HTTP-Methoden (GET, POST, PATCH) für die jeweiligen Operationen.

8.1.1. API-Dokumentation

Die API-Dokumentation findet man unter `http://localhost:3000/api-docs` (nach Start des Servers).

8.2. Testkonzept

Aufgrund der geringen Logikdichte der Einzelkomponenten (CLient und Server) und zur Vermeidung von aufwendigem Mocking wurde eine pragmatische, E2E-zentrierte Teststrategie gewählt.

- **Schwerpunkt E2E (Cypress):** Die 7 User Stories werden direkt über den Browser validiert. Dies stellt das korrekte Zusammenspiel von Frontend, API und Datenbank ohne Mocks sicher und bietet das beste Kosten-Nutzen-Verhältnis im gegebenen Zeitrahmen.
- **Unit-Tests (Vitest, Supertest):** Ergänzende Absicherung isolierter Logik. Hierbei werden Abhängigkeiten konsequent gemockt, um die Testausführung schnell und unabhängig von der Infrastruktur zu halten.

8.2.1. Teststufen

Stufe	Framework	Scope
Unit-Tests	Vitest + Supertest	Client, Server
E2E-Tests	Cypress	Gesamte Applikation

8.2.2. Testabdeckung pro User Story

User Story	Beschreibung	Testdatei
US 1	Anmeldung (Admin/User/Ungültig/Logout)	`user-story-1.cy.ts`
US 2	Technologie erfassen (Formular, Pflichtfelder)	`user-story-2.cy.ts`
US 3	Technologie publizieren (Entwurf → Publiziert)	`user-story-3.cy.ts`
US 4	Technologie ändern (Name, Kategorie, Beschreibung)	`user-story-4.cy.ts`
US 5	Technologie-Einordnung ändern (Ring, Begründung)	`user-story-5.cy.ts`
US 6	Technologien im Radar anzeigen (Kategorien, Ringe)	`user-story-6.cy.ts`
US 7	Anmeldung am Viewer (User-Login)	`user-story-7.cy.ts`

9. Architekturentscheidungen

#	Entscheidung	Alternativen	Begründung
1	**Angular** als Frontend-Framework	React, Vue.js	Vorgabe durch den Auftraggeber (Randbedingung Web-Stack).
2	**Express.js** / **Node.js** als Backend	NestJS, Fastify	Leichtgewichtig, schnell aufgesetzt und gut mit TypeScript nutzbar. Passend für den begrenzten Projektumfang.
3	**PostgreSQL** als Datenbank	SQLite, MongoDB	Bewährte relationale Datenbank mit starkem Prisma-Support und einfachem Docker-Setup.
4	**Prisma** als ORM	TypeORM, Drizzle, Raw SQL	Typsicherer Client, deklaratives Schema, automatische Migrationen und gute Dokumentation.
5	**Auth0** als Identity Provider	Eigene Authentifizierung, Keycloak	Reduziert Komplexität erheblich. Login, Rollenverwaltung und Token-Handling out-of-the-box.
6	**Tailwind CSS** + **DaisyUI** für Styling	Bootstrap, Angular Material	Utility-First-Ansatz ermöglicht schnelles, responsives Styling ohne viel Custom-CSS.
7	**OpenAPI-Typen-Generierung**	Manuell typisierter HTTP-Client	Automatisch generierte TypeScript-Typen aus der Swagger-Spezifikation vermeiden Inkonsistenzen zwischen Client und Server.
8	**Cypress** für E2E-Tests	Playwright, Selenium	Gute Developer Experience. Einfaches Setup und breite Community-Unterstützung für Webanwendungen.
9	**Docker Compose** für Deployment	Manuelles Setup, Kubernetes	Einfache Orchestrierung von Client, Server und Datenbank.

10. Risiken und technische Schulden

10.1. Risiken

Risiko	Beschreibung	Massnahme
Auth0-Abhängigkeit	Die gesamte Authentifizierung hängt von einem externen Dienst ab. Bei Ausfall oder Änderungen der Auth0-API funktioniert der Login nicht mehr.	Auth0 bietet hohe Verfügbarkeit.
Keine Server-seitige Token-Validierung	Die REST API validiert eingehende Requests aktuell nicht gegen ein JWT-Token. Jeder, der die API-URL kennt, kann Daten verändern.	Sollte dringend nachgebessert werden, leider fehlte die Zeit dafür.

10.2. Technische Schulden

Schuld	Beschreibung	Auswirkung
DB Resets bei E2E-Tests	Bei den E2E-Tests wird die Datenbank zwischen den verschiedenen Test-Files nicht zurückgesetzt.	Einbauen eines Skripts, das die DB vor jedem Test-File auf einen definierten Zustand zurücksetzt.
API-Aufrufe	APi aufrufe haben momenten keine Fehlerbehandlung (Nur Logs im Server), was zu unhandlichen Fehlermeldungen im Client führt.	Statuscodes und Fehlermeldungen im Frontend korrekt behandeln.

11. Fazit

Auf das Projekt habe ich mich sehr gefreut, um das gelernte Wissen aus dem Modul umzusetzen. Es hat mir sehr viel Spass gemacht, die Applikation zu entwickeln und die Architektur zu gestalten. Anfangs dachte ich, das Ganze wird sicher in den definierten 60 Stunden gut machbar sein, aber leider musste ich schnell feststellen, dass es doch deutlich mehr Zeit in Anspruch genommen hat.

Herausforderungen beim Technologie-Setup

Für das Projekt nahm ich den Vorschlag vom Dozenten an und dachte mir, dass ich noch einige coole Technologien verwenden könnte. Leider führte das dazu, dass ich beim Aufsetzen einzelner Technologien viel länger gebraucht habe, als mir lieb war. Dies führte dazu, dass ich nun einiges mehr an Stunden angesammelt habe als geplant.

Ein konkretes Beispiel war Prisma, welches vor kurzem auf die Version 7 erhöht wurde. Da die Version brandneu war, gab es Probleme mit dem IntelliJ-Plugin. Die Fehlersuche hat mich mehrere Stunden gekostet, bis ich herausfand, dass ich den prisma/language-server@7.0.0 manuell installieren und konfigurieren musste.

Allgemein musste ich feststellen, dass das Setup verschiedener Technologien oft zeitintensiver ist als gedacht. Dokumentationen sind teilweise nicht aktuell oder man stösst auf Fehler, für die es noch keine dokumentierten Lösungen gibt. Das war zwischendurch frustrierend, wenn man lange daran sitzt und das Gefühl hat, es geht nicht voran. Am Ende konnte ich dann trotzdem fast alles sauber aufsetzen.

Umsetzung der Anforderungen und Testing

Nachdem das Grundgerüst stand, verlief die Entwicklung sehr zielgerichtet ("straight forward"). Das Erstellen der Endpunkte und des Frontends ging zügig voran. Einzig das Formular für die Technologien, bei dem ich die Logik für Erstellung und Aktualisierung (Create/Update) in einem Element vereinen wollte, war etwas aufwendiger.

Beim Thema Testing war Cypress eine positive Überraschung. Da das Framework sehr angenehm zu bedienen ist, habe ich am Ende deutlich mehr E2E-Tests geschrieben als ursprünglich geplant. Ich bin ehrlich gesagt kein grosser Fan von Unit-Tests, bei denen man für jede Kleinigkeit unzählige Abhängigkeiten mocken muss. E2E-Tests bieten für mich einen viel grösseren Mehrwert, um die echten User Stories abzusichern.

Zukunft

Allgemein kann ich sagen, dass mir die Arbeit mit Angular sehr viel Spass gemacht hat, obwohl ich davor eher mit React gearbeitet habe. Und ich denke, ich werde Angular in Zukunft auch weiter verwenden, da es einfach sehr angenehm ist damit zu arbeiten. Es gibt viele coole Features und es ist sehr gut dokumentiert.

Das Projekt selbst ist natürlich noch nicht perfekt. Ich habe bereits damit begonnen, alles zu containerisieren. Das funktioniert zwar schon, bietet aber sicher noch Raum für Verbesserungen. Mein Ziel ist es, die Applikation testweise auf meinem eigenen Server zu deployen, um den gesamten Prozess einmal durchzuspielen und zu sehen, wie das genau funktioniert. Da ich mir

noch nicht zu 100 % sicher bin, ob das aktuelle Setup dafür schon bereit ist, werde ich dort nochmals Hand anlegen, sobald es die Zeit zulässt.

12. Arbeitsjournal

Studen Total: 80

Datum	Stunden	Tätigkeiten
04.02.2025	2	• Projekt-Stack definieren und einsenden
05.02.2025	5	• Dokumentationen initialisieren mit AsciiDoc und erste Inhalte anlegen • Projektstruktur aufsetzen • Client initialisieren mit Angular • Repo aufsetzen und initiale Commits durchführen
06.02.2025	7	• Server initialisieren mit Typescript, NodeJS und Express • Docker Compose aufsetzen für DB • Prisma aufsetzen und DB Schema anlegen • Seed-Daten anlegen und Endpunkt implementieren
07.02.2025	4	• Endpunkte definieren und implementieren • Endpunkte Dokumentieren mit Swagger
08.02.2025	8	• Auth0 implementieren • Dockerisierung der Applikation • DB Schema anpassen und Probleme mit Prisma lösen • Chapter 1 der Dokumentation schreiben
12.02.2025	8	• Radar View implementieren mit Dialog für Details • Routing implementieren • Component Struktur mit Layout aufbauen
13.02.2025	4	• Angular Service für Techologien implementieren
24.02.2025	5	• Admin Page implementieren • Tech-Formular aufbauen und Logik implementieren • Tech-Formular verbessern und Validierung hinzufügen
26.02.2025	5	• Create und Update Calls implementieren • Tech-Formular verbessern und Validierung hinzufügen

Datum	Stunden	Tätigkeiten
27.02.2025	8	• Logik für Veröffentlichungsdatum implementieren • Rolleninformationen in Auth0 anlegen • File-Struktur verbessern • Home Page implementieren • Delete Call implementieren • Server Unit-Test mit Vitest und Supertest implementieren • Client Unit-Tests mit Vitest implementieren
28.02.2025	10	• Cypress aufsetzen und commands implementieren • Cypress E2E-Tests implementieren • README's erstellen
01.03.2025	10	• Dokumentation erstellen • Fazit schreiben
04.03.2025	4	• Aufräumen und letzte Verbesserungen vornehmen